# Smart Water Filter Base

Design Document

CSE 123A - Winter 2026

Group 8

Contributors:

Logan Bossuwe
Dev Chodavadiya
Shriyan Gote
Reid Graham
Sam Lai
Pieter Nauwelaerts

May 7, 2026

# Executive Summary

This design document covers the creation of a our Smart Water Filter Base, which is a device that monitors the weight of water in a pitcher to determine the remaining water level and notify users when it is low.

Shared environments, including roommates, families, and offices, often rely on a single water filter. This can often lead to frustration if users frequently discover that the container is empty when they try to use it. This causes inconvenience, wasted time, and frustration.

Our proposed solution is a smart base that is placed under the filter and continuously measures the weight. This weight is converted into a percentage level and sent to users through a mobile app, notifying them if it gets low.

The system is composed of hardware, software, and web components.

**Hardware:**

- Load cell for measuring weight
- Amplifier for sensor communication
- WiFi-enabled microcontroller for processing and communication
- Lithium battery for long term power

**Software:**

- Microcontroller firmware for reading sensor data and sending notifications
- HTTPS server for communication between the microcontroller and the mobile app
- Website for initial setup instructions and account management
- Mobile app for displaying water level, group, and notifications

The Smart Water Filter Base is a simple, practical solution for monitoring water filter levels. It is easy to use and install, working with existing water filter pitchers without requiring any modifications. By providing real-time updates on water levels, it helps users avoid the inconvenience of discovering an empty pitcher when they need water.

# Contents

# 1 Introduction

## 1.1 Problem statement

Living with roommates can sometimes mean your filtered water device is empty when you need water. This can be very frustrating and leave you needing to drink tap water while other roommates get to drink cold refrigerated and filtered water.

## 1.2 Need Statement

**We want a way to know if the container is out of water when we need water.**

- Avoid having no water left in addition to everybody using the water filter, not knowing there is no water left.
- Need a way to know the current water level of the pitcher so that it isn't empty without everyone knowing it's empty.
- We need to know if the container is empty, in order to not run out of clean fresh water.

## 1.3 Goal Statement

- Keeping water level known at all times, while informing everybody in the household if the water level runs too low.
- Create a system to inform users if the container is empty.

**If the container is empty, inform users.**

## 1.4 Personas and Users

**This is intended for shared households that rely on a shared filtration water container.**

- College Roommates
  - John Doe, 20 years old, Student at UCSC. John lives in a dorm with two other roommates, and they all rely on a shared water filter for drinking water. Occasionally, John comes out to get water only to find that the water is empty. This creates frustration and delays as John has to choose between missing the bus to refill water or getting to class on time while being thirsty.
- Family
  - Jane Smith, 39 years old, working mother. Jane lives with her husband and two children. Jane gets home after a long, exhausting day of work and is feeling thirsty from the drive home. She goes to the shared water filter to find that it is completely empty. Jane feels frustrated with her stay-at-home husband and two children for not refilling the water filter. This causes her to lash out and yell at them until they all cry, creating a hostile environment in the household.

- Office
  - Joe Micheal, 30 years old, works with Excel spreadsheets. He has very limited time to refill his mug, and his boss, George, never refills the water filter. As a result, Joe's efficiency has substantially decreased at his desk.

## 1.5 Research into Existing Designs and Products

To understand where our design fits, we researched three existing products that partially meet the same need.

### 1.5.1 Withings Body Smart

The Withings Body Smart is a consumer Wi-Fi scale that measures weight, body fat, muscle mass, and water percentage using strain gauge load cells and bioelectrical impedance analysis. It syncs data automatically to the Withings smartphone app, where users can track trends and set goals. While it offers a polished user experience, the system is entirely closed—users must rely on Withings' proprietary app and cloud service, with no way to self-host data or customize the interface.

### 1.5.2 Wyze Scale Ultra

The Wyze Scale Ultra takes a similar approach but adds a 4.3-inch color display directly on the scale, so users can view 13 body composition metrics without needing their phone nearby. It supports Wi-Fi syncing to the Wyze app and integrates with third-party fitness platforms like Apple Health and Google Fit. Like the Withings, it is a closed system that cannot be modified or repurposed beyond its intended body composition use case.

### 1.5.3 Hackster.io ESP32 MQTT Weight System

On the DIY side, a Hackster.io project by The Embedded Things implements an ESP32-based weight measurement system using an HX711 amplifier and load cell. It features MQTT communication for real-time data streaming, multi-sample averaging for noise reduction, and automatic tare on startup. This project demonstrates strong signal processing but requires an external MQTT broker and separate dashboard software, adding complexity that our self-hosted approach avoids.

### 1.5.4 Where Our Design Fits

Unlike commercial smart scales, our product is specifically designed for frequent monitoring of the fill level of a water pitcher. The system emphasizes event-based functionality, such as notifying the users of change in water level and when it needs to be refilled. Unlike the Hackster.io project, which focuses on the scale functionality and data transmission, our design features an end-to-end system including measurement, data storage, a web server, and a mobile app interface. A mobile app is much more accessible and easy to use than fully self-hosted or local solutions.

## 1.6 Sustainability Statement

Our project promotes sustainable water consumption habits in shared households. Without knowing the current water level, users often open the fridge only to find an empty filter, leading them to either waste time waiting for it to refill or even use one-time plastic bottles. By providing real-time water level monitoring and low-level alerts, our system encourages refilling and reduces the likelihood of users turning to less sustainable alternatives. Over time, this helps households maintain consistent use of their reusable water filter rather than abandoning it out of frustration.

# 2 Design

## 2.1 Design Objective

Through the goal statement, it is needed that water level is known at all times to the user. To approach the goal, two parts needed being what hardware is needed to get accurate readings and what software is needed to communicate the information to the user.

### 2.1.1 Hardware

We will be collecting data from the users through the weight of their water filters. All of the hardware will be contained in a plastic base which the filter will be set on. This will provide space for all of the internal electronics to be enclosed and kept dry in the humid refrigerator environment. The internal components will need to be sealed in completely to prevent them from getting wet.

The internal hardware components need to complete a few simple tasks. Detail about specific design choices will be explained in later sections focusing on specific component requirements. The following is a list of requirements for the hardware:

- Collect real time weight data
- Package weight data in an HTTPS packet and send it through Wi-Fi
- Work on battery power for long periods of time
- Charge the internal battery using a power port
- Have Wi-Fi and Bluetooth for initial setup and active use.

With these requirements in mind, our design uses a few internal components. Most components can be mounted to a custom designed circuit board. Others are connected with built in pins to the circuit board.

### 2.1.2 Software

The requirements for the software design involve a few segments. They can be split into categories based on which components they interface with. Starting with the microcontroller code, this code needs to be capable of reading in values from the amplifier which provides serial data through an analog to digital converter. This serial data needs to be converted to a weight value and sent to the data collection server using a secure protocol. The microcontroller should also be capable of sending battery status values including a percentage estimate of the battery charge.

The software from the microcontroller will send packets containing info to the data collection server. This server must be capable of mapping incoming data to related user accounts that are subscribed to the specific device. Using a unique device ID and authentication token given by the server, incoming data can be assigned to the correct point. The device ID should be sent at the time of setup and will not change through operation. The server should keep tables containing users and their subscribed devices. This server needs to be secure and able to handle the traffic of a large number of devices communicating in real time.

Once the data is at the server, the mobile application used by users should get up to date information about the status of their filter and active fill value. This data should be updated close to real time in order for features such as notifications to work as intended. The mobile application must be capable of authenticating users using username and passwords unique to the user. The application must include information about each subscribed base, and have accurate information about their status. The app should include a notifications system that will alert users that the fill value has passed below the threshold. The application should communicate using secure protocols to prevent user data from being sent unsafely.

## 2.2 Aesthetic Prototype

Our current prototype consists of two 3D-printed circular base plates sandwiching a 75mm bar-type strain gauge load cell. Each plate is 110mm in diameter and 8mm thick, printed in PLA/PETG. The top plate has countersink pockets on the bottom face for M4 bolts that secure it to one end of the load cell, and the bottom plate attaches to the other end, sitting flat on the surface. The load cell wires route out to the HX711 amplifier and microcontroller on a nearby breadboard.

### 2.2.1 Exploded View

Figure 1 shows the exploded view of the load cell assembly with all components labeled.

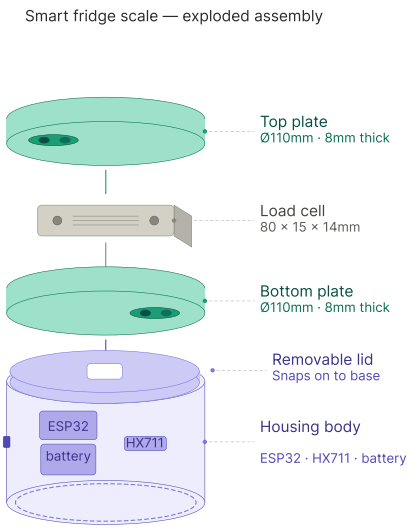



Figure 1: Exploded view of the load cell assembly showing top plate, bottom plate, load cell, and M4 hardware.

### 2.2.2 Base Plate Schematic

Figure 2 shows the design schematic of the base plate with dimensions and screw hole placement.

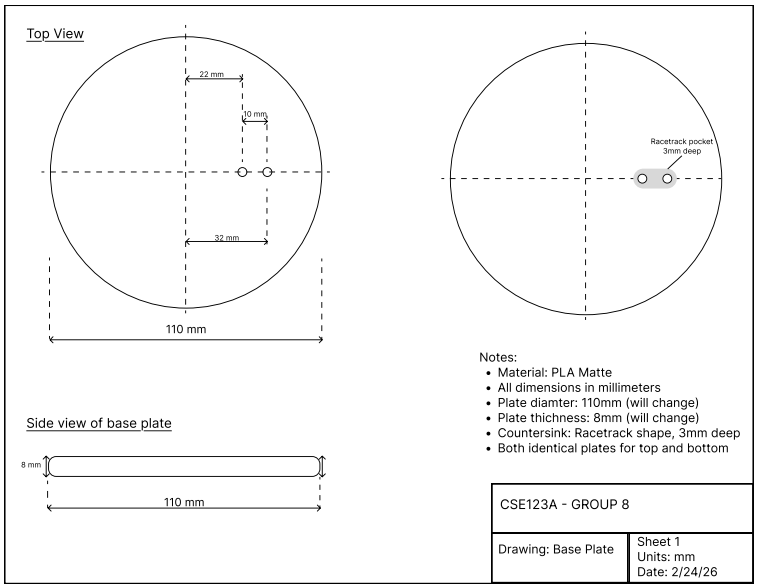



Figure 2: Design schematic of the base plate.

### 2.2.3 Rendering

Figure 3 shows how the complete system looks when assembled with a water filter pitcher placed on the scale platform.

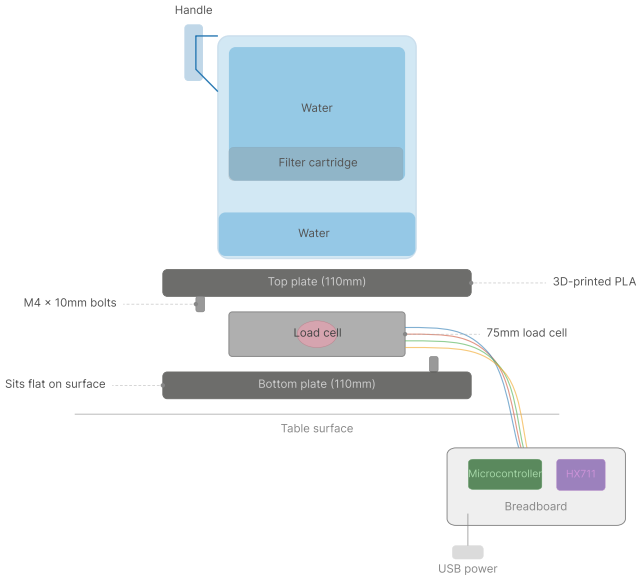



Figure 3: Diagram of the full system assembly showing the water filter pitcher resting on the load cell platform, with wiring to the microcontroller and HX711 on the breadboard.

### 2.2.4 Physical Prototype

Figure 4 shows the assembled prototype from a side view, with the load cell mounted between the two plates and wired to the breadboard.

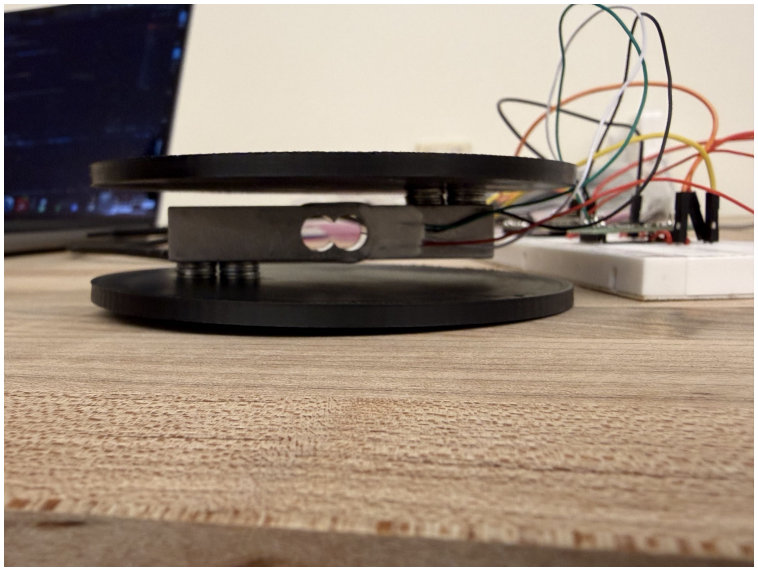


Figure 4: Side view of the assembled prototype showing the two base plates, load cell, and wiring to the microcontroller.

## 2.3 Design for Manufacture

The design is intended for large scale manufacturing using a custom PCB. This custom PCB includes all required components such as the CPU, RAM, and Flash storage required to function as a microcontroller. It will also include power contacts for the battery, a power management unit for charging the included battery, and components like the WiFi antenna. The PCB design is an outline of what is required. Selected components can be changed and the circuitry altered as required if parts are unavailable.

The PCB will be mounted inside the 3D printed base and all other components will be contained within the base or related plates. The product is intended to be useable, and rechargeable without having to take apart or disconnect any internal components. This is so that it can be properly sealed from humidity to protect the sensitve electronic components.

### 2.3.1 3D-Printed House

The base assembly consists of three 3D-printed parts: a top plate, a bottom plate, and a compartment at the bottom for the electronics. All parts are printed in PLA. The top and bottom plates are both 110mm in diameter and 8mm thick, with M4 screw holes and racetrack-shaped pockets for mounting the load cell. The plates went through multiple iterations to resolve flatness issues, which affected the stability of the load cell.

The compartment is a circular open-top tray with a 116mm outer diameter and a 110mm inner diameter, sized so the plates sit flush inside it. There is a resting shelf for the bottom plate to sit on, and tabs slightly above it so that when the plates are turned, they are held in place.

In the assembled stack-up, the compartment sits below the bottom plate so that the load cell remains free to flex under load while the electronics remain enclosed. Since the

compartment is open at the top, the top plate is slightly wider, serving two purposes: to provide a cover for the compartment, and to be large enough for various sizes of pitchers to be placed on top of it.

### 2.3.2 Load Cell Assembly

The 75mm bar-type strain gauge load cell is sandwiched between the two plates using four M4 x 10mm bolts—two through the top plate into one end of the load cell, and two through the bottom plate into the other end. This creates a cantilever arrangement where weight applied to the top plate causes measurable strain on the load cell. The assembly requires only an M4 hex key and can be taken apart and reassembled in a few minutes.

### 2.3.3 Electronics

The electronics will be contained inside a separate section of the 3D Printed base. The top section contains the load cell. The bottom section will have all of the mounting for the PCB, battery, and charging input. The wires for the selected load cell will come down through the bottom mounted plate through a small hole, then be connected to pins on the custom PCB. Wiring for the charging port will also mount to connector pins on the PCB. Wire length should be minimized to reduce the movement of internal components during shipping.

### 2.3.4 Power

A battery is the best method of providing power for this product. In order to keep the user from having to repeatedly charge it and remove it from their fridge, a battery with a sufficiently large storage of milli-amp hours is necessary. Also included in the design is a method of charging the battery that does not require taking the device apart. The charging port will be able to charge the battery and keep the device powered so that users won't lose connection to their device.

## 2.4 Amplifier

In order to get readings for weight from the load cell, it is most likely required to use an amplifer if the chosen load cell is analog.

### 2.4.1 Digital Converter

Software will need to be implemented in order to read data, analog gives the signals in terms of bits that need to read and converted to either raw data of calibrate straight to grams.

### 2.4.2 Average Sampling

Sampling without averaging can lead to lots of noise. Average sampling allows the accuracy of weight to be more accurate.

### 2.4.3 Tare

For handling offset, it is required to create to create a baseline state value that represents an empty pitcher. It means future data is can be compared to the baseline value to get the water weight readings.

## 2.5 Block Diagrams

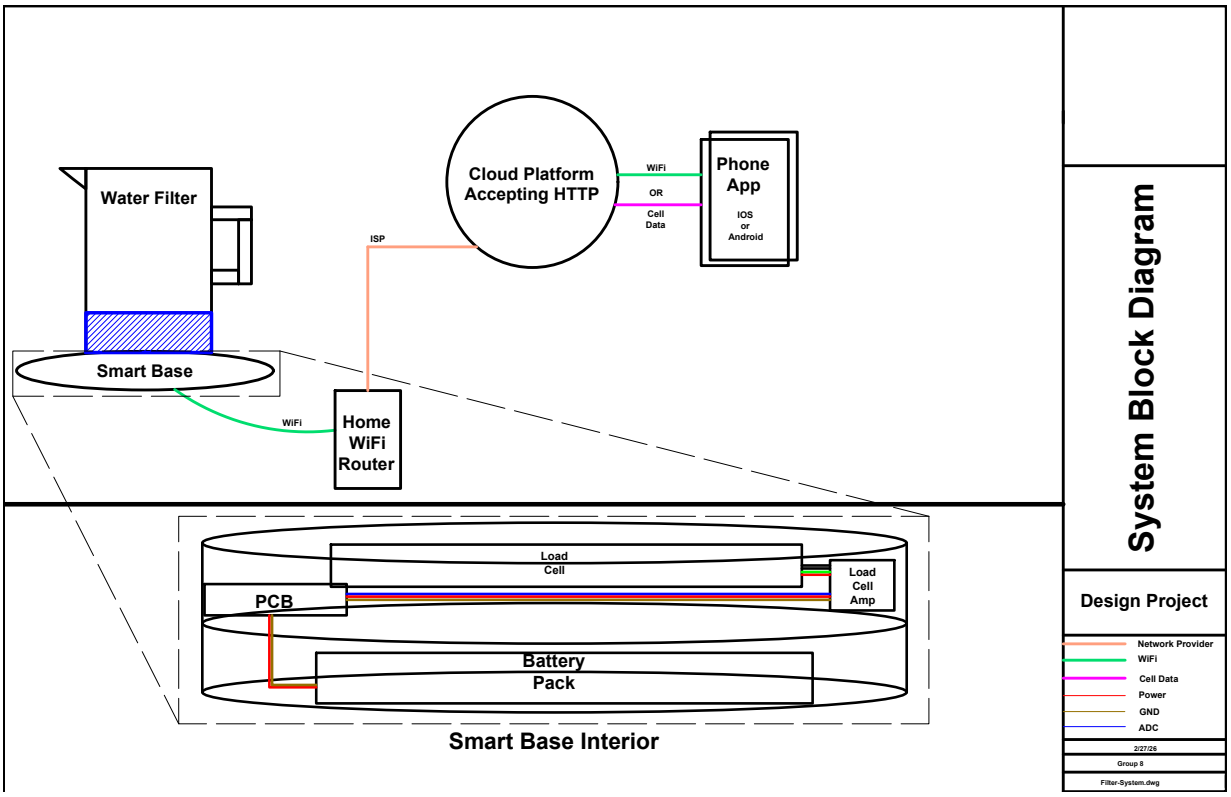



Figure 5: The system block diagram represents the pieces of the device and how they are interconnected.

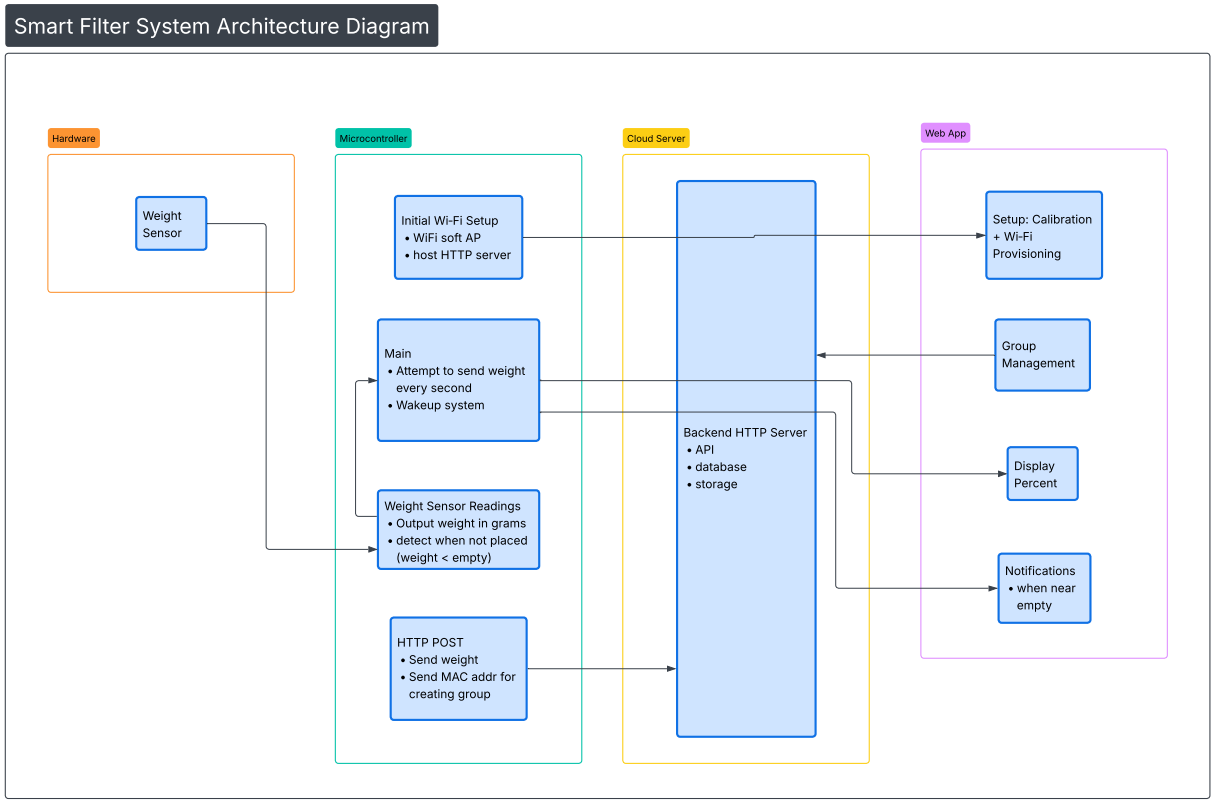



Figure 6: The basic architecture of our project, including the modules and submodules of hardware and software, along with how they interact and communicate with each other.

## 2.6 Wiring Diagrams

The wiring diagram for this design includes four pieces. The batter, microcontroller, load cell amplifier, and load cell. These parts all tie together with different wires connected to specific pins on each device. Each pin is labeled and each wire color coded. See Figure 7.

## 2.7 Custom Design PCB

In order to produce large amounts of product, the cheapest and fastest method is to use a custom circuit board. This circuit board would ideally integrate all necessary components with as little extra work as possible. We provide an example design using available components. These components can be swapped with other similar types, as long as their main functionality is still met.

### 2.7.1 Processor

The selected processor is the ESP32-C3-MINI-1. This processor is widely available and was selected for the following reasons. It includes 2.4GHz Wi-Fi and Bluetooth antennas in the packaging. This is ideal for our design as it reduces the number of required compo-

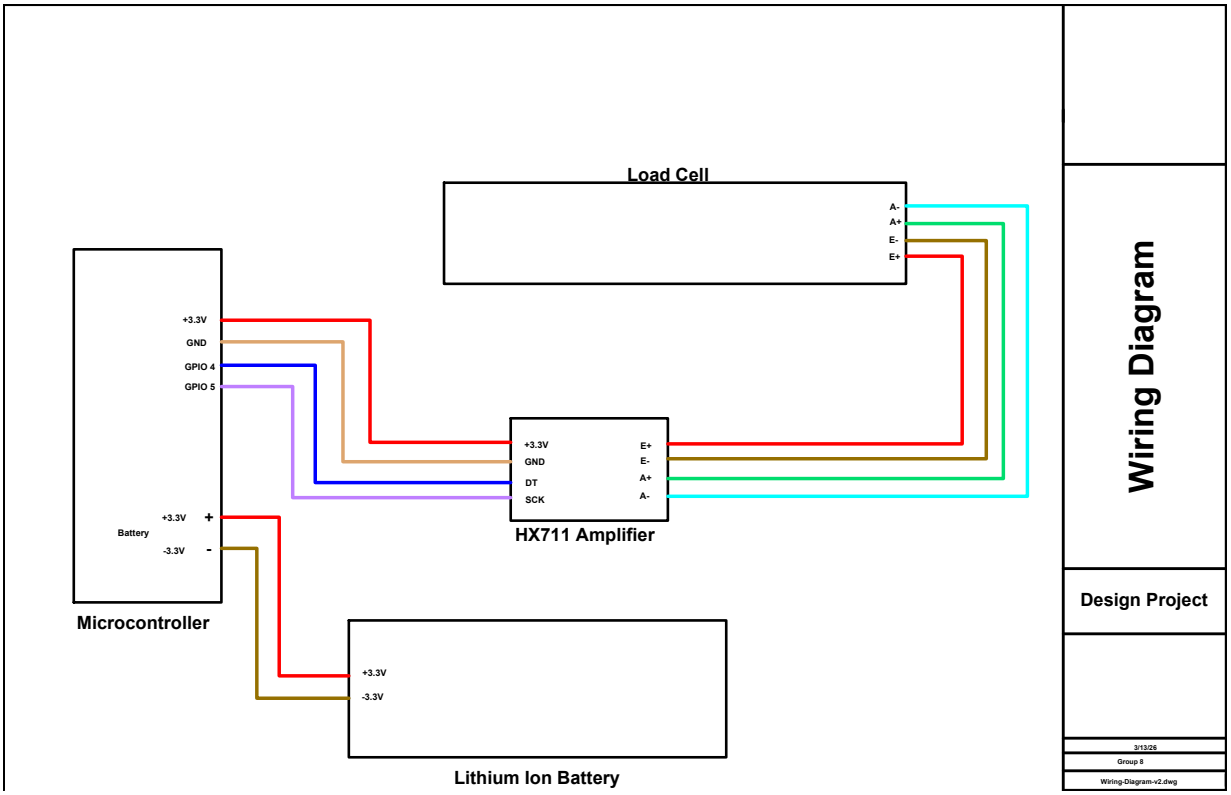



Figure 7: Wiring Diagram of Components Included in System Design

nents. Adding a separate antenna is possible but would require a different chip selection that includes the required pins for Wi-Fi and Bluetooth.

The pins we use are the GPIO 4 and 5 pins. On this specific chip they are the ADC channels 0 and 4 respectively. These pins allow us to read data from our amplifier chip, and through the code we are able to interpret that as a weight value. These pins are required for this setup. Using another method like I2C or UART might be possible but would require a complete redesign to match our requirements. Analog to Digital pins are the simplest method for this design.

The power supply for the chip is provided through the 3.3v pin. We selected a power method that was capable of powering both the amplifier board and the ESP chip. Both can be powered with 3.3 volts. For most of their operational time, the ESP chip will be in standby mode which consumes very little power. The most power intensive time is when transmitting via WiFi as that has a tendency to spike the power consumption. In order to handle this, a power management circuit is required to prevent spikes in current.

### 2.7.2 Power Components

For this design we provide the MCP73871 as an example chip selection. This is a power management IC that is capable of switching between a USB or AC/DC power source, and a lithium ion battery. When the USB or AC/DC source is providing power, it will power the circuit load and charge the lithium ion battery. This chip selection includes a lot more

features than our design takes advantage of, meaning room for expansion of features if desired.

The requirements a selected chip needs to include are the following. First, it needs to output the correct voltage for the selected amplifier and CPU combo. If not using an amplifier but some other method of data collection, then it needs to be able to output the required power for that method. Second, it needs to take two inputs, a USB or AC/DC input, and a lithium ion battery input. Third, it needs to be capable of charging the lithium ion battery, otherwise the product will only last a single charge and then not be useful to the customer.

Our selection for this design is to use a USB-C charging receptacle. This would allow us to place a floating port anywhere on the base that would be cheap to supply the cables to users. USB-C is a simple standard that can supply enough power if selecting a part that has only power pins enabled. The selected USB-C port in our designs contains the positive and negative power pins and nothing else. This meets the requirements for the selected power management chip. In order to use the design the way we intend you must select a USB-C or other power method that provides a positive and negative feed to the power chip.

### 2.7.3 Load Cell Amplifier

The load cell amplifier is a required component for this specific design. Changing the design significantly to where it uses a different load cell than we had intended will require a redesign of this specific circuit component as it is only required for the load cell method selected. The load cell we describe requires a board which amplifies the microvolt output it provides. This chip selection is very important for processing the sensitive weight data produced by a load cell.

The amplifier board must be powered from the same circuitry as the main chip to keep compatibility. In this design we specify 3.3 volts as the power source, but selecting a chip with a 5 volt or higher input would mean the selected amplifier chip must also be capable of 5 volt power, or wired up using a voltage divider circuit to get the required voltage.

The output of the amplifier board is used by the main CPU and program to determine the current weight placed on the sensor. In our design we specify that output going to analog to digital converter pins meaning the output of this device must be an analog signal, or something that can be interpreted by GPIO pins.

The inputs for this board come from four pins. We have designed the board so that it can be connected through a 4 pin JST connector. Any four pin connector that is small in form factor and can be soldered to the board horizontally, to lower the height requirements, is the best choice for this design. Keeping the height of the board minimal will mean the plastic base can be smaller and take up less space in a customers fridge. A 4 pin connector keeps the product from requiring soldering wires directly to a board, rather the wires for the load cell can be placed in a connector housing and plugged in directly. In order to seal the product well and prevent movement during shipping, a connector with a locking mechanism is the best choice for the design.

### 2.7.4 Load Cell

The load cell selection makes a big difference in how much the product can handle. For our prototype and our design we selected a 10 kilogram load cell which should be capable of weighing almost all commercial water filter pitchers. This selection means the most compatibility for the most consumers. Selecting a smaller load cell would limit the number of customers who can purchase it if they have larger pitchers capable of holding more than 4 liters of water. Even if the majority of consumers have smaller pitchers, we hope to provide the most availability.

The load cell in our design uses four wires to provide readings through a load cell amplifier. The design involves connecting the four wires of the load cell to the board using a 4 pin JST connector. The specific connector choice is described in the amplifier section. The boards will be physically separated by plates that have a small hole for cable routing. The cables should run from the load cell, through the hole, to the pin connection point on the custom PCB.

## 2.8 Microcontroller WiFi Provisioning

In order for the microcontroller to connect to the home network, the user must be able to send the WiFi credentials to the device. This is called WiFi provisioning. In our design, the user connects to the device through BLE and sends the data through the mobile app.

### 2.8.1 On Boot

When the device starts, it will attempt to load saved WiFi credentials and connect to the network. If this fails, it will enter provisioning mode. In our design, the MCU hosts a BLE access point that the user connects to through the mobile app. Once connected, the user will send the WiFi credentials through BLE to the device. The mobile app will also request a unique authentication token from the server for the device to use in the future, which will also be sent over BLE. The MCU will then attempt to connect to the specified WiFi network using the provided credentials. If the connection is sucessful, the device will save the wifi credentials to non-volatile storage and reboot.

### 2.8.2 Lost Connection

If WiFi connection is lost for any reason, the device will enter recovery mode. It will repeatedly attempt to reconnect, exponentially backing off over time to save resources. If connection is restored, the device will go back to normal functioning.

### 2.8.3 Manual Reset

If the user changes WiFi networks or needs to restart the device for any other reason, they can use the reboot button found on the smart base.

## 2.9 HTTPS Capabilities

The smart base must be able to securely communicate with a remote server or cloud service using HTTPS. This is necessary to send data about the water level, device identication data, and authentication data. It also must be able to recieve authentication data or tokens from the server, which it will use for future communications.

The smart base must support:

- HTTPS client connections using TLS 1.2 or higher
- standard HTTP methods for bi-directional communication (e.g. GET, POST)
- server certificate validation to ensure secure communication
- reliable error handling for failed connections or invalid certificates
- ability to reestablish connections if the network is temporarily unavailable
- secure storage of any necessary credentials (e.g. API keys) for authenticating with remote servers
- DNS resolution to connect to servers by hostname rather than IP address

The smart base must be able to securely transmit data to a remote server or cloud service for storage and analysis. This requires secure HTTPS client functionality, with the ability to both send and recieve data. Additionally, the smart base should be able to handle common network issues or errors. Although DNS resolution is not strictly required depending on the implementation, it is strongly encouraged to allow for flexibility and scalability in the system design.

## 2.10 Web Application

The web application is the user-facing interface for the smart water-level monitor. The design goal is to provide a clear real-time estimate of remaining water in the container, allow user calibration for different pitcher sizes, and send refill notifications when the water level becomes low.

The current design is a React application deployed on Vercel, with a Supabase backend for data storage. The frontend displays the latest water level reading as a percentage, along with a visual representation of the water level in the pitcher. Users can calibrate the system by setting empty and full reference points, which allows for accurate level estimation regardless of pitcher size or sensor placement. The application also includes functionality to send push notifications to users when the water level falls below a certain threshold.

### 2.10.1 System Architecture

The deployed architecture has three coordinated layers:

- **Frontend dashboard:** Displays the latest reading, computes estimated level percentage, and status indicators.

- **Vercel backend:** Receives sensor uploads and notification subscriptions.
- **Supabase Storage:** Stores water readings, calibration parameters, and browser push subscription tokens.

The frontend was designed using the following languages: HTML, CSS, and JavaScript. It utilizes a React framework for building user interfaces. The backend is implemented on Vercel, written in JavaScript (Node.js), with API endpoints for data ingestion and notification management. We are using Supabase, which provides a PostgreSQL database for persistent storage and real-time capabilities.

### 2.10.2 Frontend Design and User Experience

The current dashboard is minimal and task-focused. Core interface elements are:

- A large water-level visualization (pitcher graphic with animated fill height).
- A numerical percentage readout for quick interpretation.
- A water-present/empty status indicator.
- Metadata showing last update timestamp, measured weight, and battery voltage (when available).

Users can quickly assess whether refill action is needed without interpreting raw sensor values.

### 2.10.3 API Endpoints

The application has two primary API endpoints:

- **POST /api/ingest:** Receives sensor readings (device ID, weight, battery) from the microcontroller and stores the data in Supabase. If the weight is below a configured threshold, triggers a low-water notification to registered users.
- **POST /api/register-token:** Registers browser push subscriptions from the frontend and saves them in the database.

Sensor data is sent to the ingestion endpoint with an API key, validated, and inserted into a table on Supabase. The dashboard reads the newest value from Supabase on a polling interval and renders the latest state. The token registration endpoint enables user devices to register for alert notifications. When invoked, it associates a device with a user account and stores the device token in a table on Supabase for later use in push notification delivery.

### 2.10.4 Supabase Tables

Supabase serves as persistent storage for both backend APIs and the frontend dashboard state. The ingestion and token-registration endpoints write data to Supabase using a service-role key, while the frontend dashboard queries the same tables to render the latest readings and calibration settings.

The current deployment uses the following tables:

- `water_readings` — Stores time-series sensor uploads from the microcontroller. Each row represents one sample and includes the device name, weight in grams, and battery voltage in millivolts (e.g., `device_id`, `weight_g`, `battery_mv`) with a timestamp (`created_at`).
- `notification_tokens` — Stores browser/device push tokens used for low-water alerts. Tokens are inserted or upserted on registration and are later queried by the ingest pipeline when a low-water event occurs.
- `calibration` — Stores user calibration references for level estimation. The dashboard upserts a single calibration record (using `id` constant) containing empty and full reference values (`empty`, `full`), then reads the latest entry when computing displayed percentage.

### 2.10.5 Calibration

To support different pitcher geometries and sensor offsets, the application includes user calibration controls:

- **Calibrate empty:** Stores the current reading as the empty reference.
- **Calibrate full:** Stores the current reading as the full reference.
- **Reset calibration:** Clears stored values and falls back to default constants.

The level estimate is computed by linearly mapping measured weight between empty and full references, then clamping the result between 0% and 100%. This makes the UI behavior predictable while still allowing practical adjustments.

### 2.10.6 Push Notifications

The push notification feature is designed as an event-driven subsystem that alerts users only when refill action is needed. The implementation uses standard Web Push protocols with VAPID authentication and is integrated into the ingest API used for sensor data.

**1) Subscription Registration** When the dashboard is loaded, the browser UI has a button that the user can click on to request notification permission. Clicking on the button triggers a pop up that asks the user to allow/disallow notifications for the website. If permission is granted, a service worker is registered and a Push API subscription is created (or reused if it already exists). The subscription object is sent to the backend endpoint and stored in the `notification_tokens` table in Supabase.
This registration path has two design goals:

- maintain a durable list of reachable user devices
- avoid duplicate subscriptions by upserting on token value.

**2) Alert Trigger Condition** Sensor readings are received by the ingestion endpoint and stored in `water_readings` table. For each new reading, the backend computes current and previous water-level percentages and checks for a low-level threshold crossing. The low-water threshold is currently set to 20%. This design prevents repeated notifications while the level remains continuously low.

**3) Delivery** If the trigger condition is true, the backend loads all stored subscriptions and sends a Web Push payload to each valid endpoint. The payload includes a:

- notification title
- notification body
- structured data (event type and level percentage) for client-side handling.

The service worker receives the push event, displays the notification, and handles click interaction by focusing an existing app tab or opening the web page.

# 3 Evaluation

## 3.1 Functional Prototype

The prototype consists of weight sensor readings sent to an HTTP server via POST requests. The web application displays the latest measurements in a simple dashboard, allows users to calibrate the empty/full weights of the water pitcher, and has a notification feature that works on web browsers.

### 3.1.1 Hardware

The plates used above and below the load cell are made for functional prototype purposes to fit the need of measure weight readings rather than handling various water filter pitchers. Temporary HTTP server made to display and mimick how data is shown/handled.

### 3.1.2 Web Application

The current web application uses a clean, single-page dashboard layout. It is currently hosted at `https://cse123a-project-6a3s.vercel.app/`. It shows the water level as both a percentage and a pitcher graphic. The pitcher graphic fills or empties based on the latest sensor reading, so users can quickly see how much water is left. A status message shows whether the water pitcher has water or is empty, and recent update details are shown to confirm data is being received correctly.

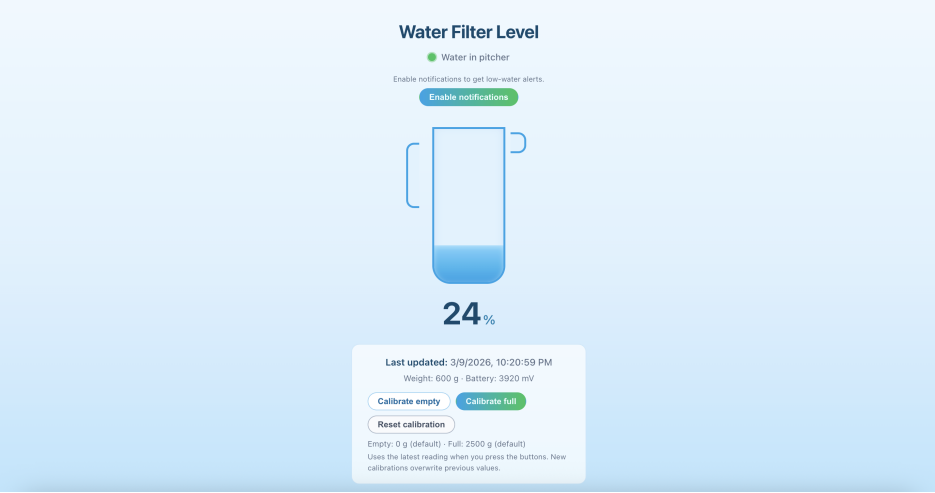



Figure 8: Current Web Application Prototype Interface

The web application works as follows:

1. The microcontroller sends a new sensor reading to the server.
2. The server stores the reading and makes it available to the dashboard.
3. The dashboard fetches the latest value and updates the displayed percentage and water-level visual.
4. Users can calibrate the system by pressing "Calibrate empty" when the latest value is an empty pitcher and "Calibrate full" when the latest value is a full pitcher.
5. If the value is below the low-water threshold, the app sends a refill alert to users who enabled notifications.

To calibrate the system, users will place an empty pitcher on the base and press "Calibrate empty" to set the empty value. Assuming the latest updated weight was 600 grams, you can see in figure 9 that under the calibrate buttons, empty is now set to 600 grams and the water level is set to 0%.

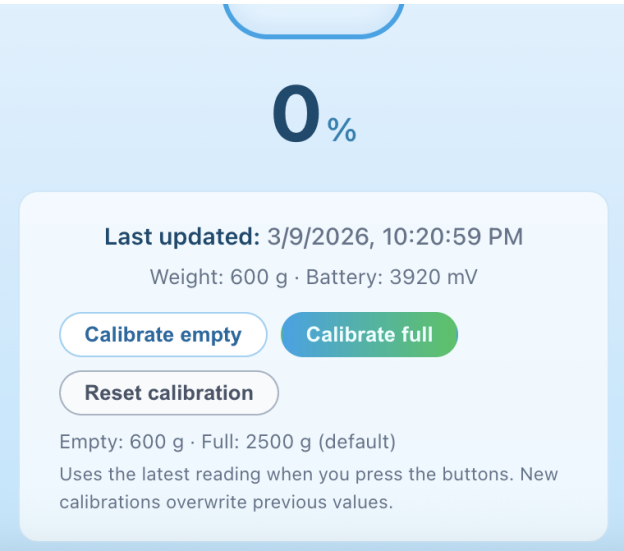



Figure 9: Web Application After Calibrate Empty

After calibrating the empty value, users can fill the pitcher to the full level and place it back on the base. Pressing "Calibrate full" will set the full value. Assuming the new latest updated weight was 1500 grams, you can see in figure 10 that under the calibrate buttons, full is now set to 1500 grams and the water level will be adjusted to 100%. Users can also reset the calibration values at any time by simply pressing "Reset Calibration" button. This sets the empty and full values back to the default values, 0 grams and 2500 grams respectively.

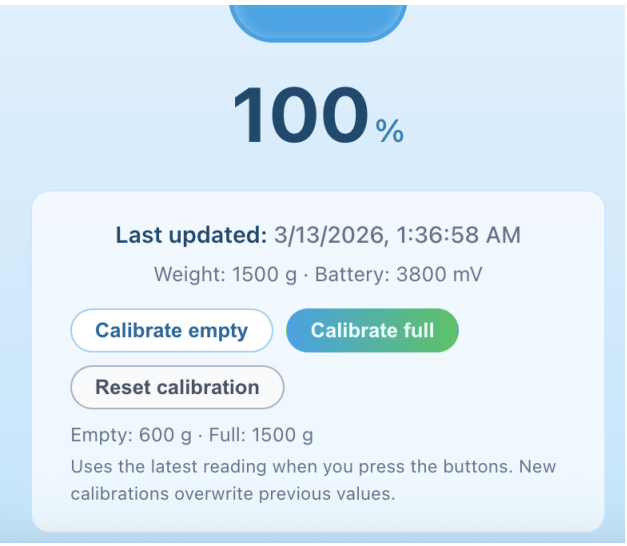



Figure 10: Web Application After Calibrate Full

To subscribe to notifications, users can click the "Enable Notifications" button. This will trigger a permission prompt in the browser as shown in figure 11.

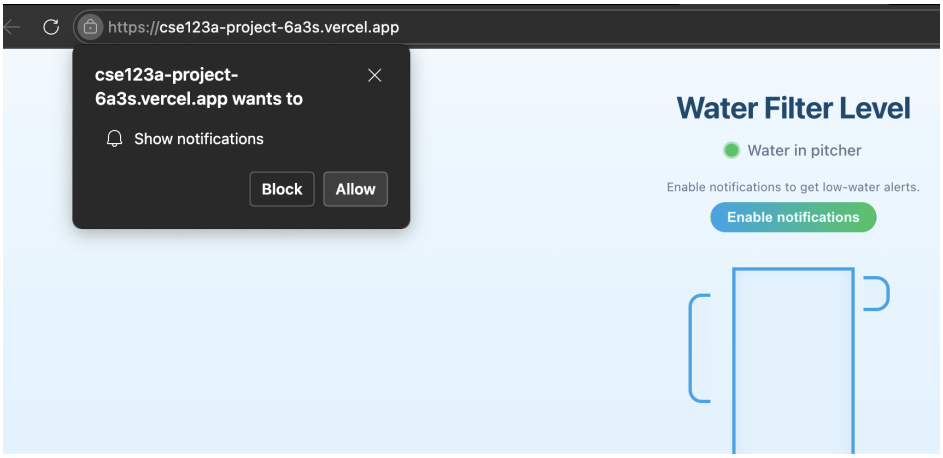



Figure 11: Web Application Notification Permission Prompt

If users allow notifications, they will be subscribed to receive alerts when the water level drops below the low-water threshold. The UI will also update to show that notifications are enabled under the water status indicator as shown in figure 12.

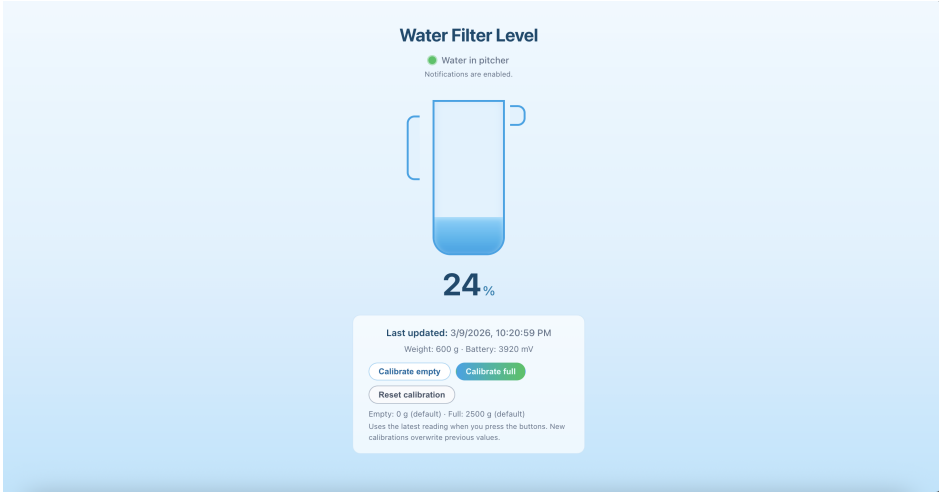



Figure 12: Web Application with Notifications Enabled

When the water level crosses the low-water threshold, which is currently set at 20%, the browser will display a refill notification popup as shown in figure 13. Clicking on the notification will take users to the dashboard where they can see the current water level and recent updates.

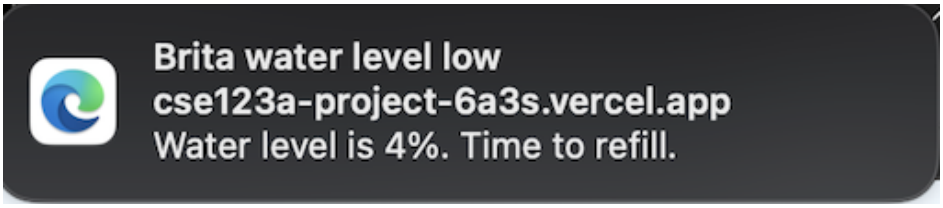



Figure 13: Low-Water Notification Popup

## 3.2 Testing

Testing is a critical phase in the development of the smart water-level monitor, ensuring that all components function correctly and integrate seamlessly. We will conduct testing at multiple levels, including unit testing for individual modules, integration testing for combined components, and end-to-end testing for the entire system.

Our local tests are linked inside the repository under Code/'Links to Test Cases.md'.

### 3.2.1 Microcontroller

When creating the tests for the microcontroller code, it's important to consider what a user behavior might look like, but also important to understand what level the behavior could get to. How likely is it that a user is putting a full filter on, then an empty one, and back to full, in a matter of milliseconds? How capable is this controller of dealing with sharp spikes and changes? The testing plan covers both simulated behavior for quick and sharp event behavior, as well as a more practical testing plan for typical user behavior.

In order to verify the function of the code reading from the Wheatstone bridge sensor, we must verify that the logic itself works the way we think. There are multiple parts that require logical verification.

First, the sensor data will come in with some small variation depending on where the weight is placed on the base, and because ADC values can vary slightly without any other input. Sensor reading code must also account for the fact that data can change very quickly, so testing the logic through repeated, high frequency changes, can help show its robustness. There is also the issue of whether data inputted to the code is read in correctly through the bit shift operations, so checking the shift operations for correctness is important.

Testing the microcontroller code in this design involves a separate file that contains simulated HX711 behavior. This behavior is passed into the functions used on the microcontroller, which provide an output back to the test file. These outputs are verified against expected outputs and if they are the same or within a certain level of tolerance, then we can say the code passed the tests.

To test the behavior that would be seen for a typical user, we must have standard objects that we use for the different levels of weight. A large weight, a small weight, and no weight are the simplest form of this. For example, fitting with the project, say a water filter with 100mL of water, versus a water filter with 3L of water. These two objects have drastically different weights so putting it on the scale and viewing how the code responds could represent a typical user behavior of taking the empty filter, filling it, and returning it to the base.

### 3.2.2 Web Application

When testing the web application, we focus on both normal use and edge cases. A normal flow is a user opening the app, calibrating once, and checking the water level. Edge cases include repeated page refreshes, pressing calibration buttons several times, or receiving rapid updates from the sensor. We also test edge cases like if the water level goes beyond expected limits. We want to ensure the app can handle these scenarios without crashing or showing incorrect information.

For frontend unit tests, we check important UI parts one at a time, such as calibration buttons, status messages, and notification prompts. These tests verify that components render correctly, user clicks update state as expected, and displayed values stay consistent with incoming data. Because sensor readings can change often, we also check that the UI updates smoothly and does not show stale values.

For integration testing, we verify that the frontend and backend communicate correctly. The app sends requests for actions like token registration and sensor data ingestion, and we check that the API returns the expected results. We test both valid and invalid payloads to make sure errors are handled clearly and data is not updated incorrectly.

Finally, end-to-end tests cover complete user workflows from start to finish. These include loading the app, calibrating, monitoring water-level changes, and receiving notifications at threshold events. If these tests pass under both normal and adverse conditions, we can conclude the web application meets functional testing goals.

## 3.3 Web Application Testing

The web app test file is designed to check the main user-facing behavior of the React application in a clear and controlled way. The tests render the app with mock data so we can verify behavior without needing live hardware or external services. The test file is organized under the test directory within the source directory, along with the file `sampleWaterValues.js` that contains mock data. The test file uses the React Testing Library to render components and simulate user interactions, and Vitest for mocks, assertions, and test organization.

### 3.3.1 Basic UI Rendering

This section verifies that the interface renders correctly in both empty and normal states. It checks the message shown when no reading exists, then confirms that water-level data, weight, and battery metadata are displayed when readings are available. It also checks that the displayed percentage matches the expected calculation.

### 3.3.2 Calibration Actions and Edge Cases

This section tests user interaction with calibration controls. The test suite clicks the calibrate empty, calibrate full, and reset buttons, then verifies that the expected calibration upsert operations are made. This section verifies calibration logic for both standard and edge conditions. The tests confirm that custom empty/full calibration values are used, and that percentage output is clamped correctly. Readings below empty are shown as 0%, readings

above full are shown as 100%, and invalid ranges where full is less than or equal to empty also resolve to 0%.

### 3.3.3 Push Notification Behavior

This section tests notification flows with mocked browser APIs. It includes unsupported-browser behavior, denied permission behavior, and successful permission behavior. In the successful case, the test verifies service worker registration and push subscription setup.

### 3.3.4 API-Related Checks

This section covers API behavior that is directly tested. Specifically, when notifications are enabled successfully, the app is verified to send a POST request to the token registration endpoint. Supabase reads and calibration writes are mocked so request behavior and state changes can be checked without live backend dependencies.

## 3.4 Testing HX711

To test the code written for this project, we defrost had to create a simulated version of the sensor. Regardless of what the sensor reports we want to make sure that all of our math and interpretation of incoming data is correct. To do this, the functions were adapted to either take input from GPIO pins, or from a specific test file.

### 3.4.1 Test File

Inside the test file are adapted versions of each function used by the main HX711 file. These include the following:

- hx711_set_sck
- hx711_get_dout
- read_raw
- get_raw_weight
- tare
- get_grams

With each of these functions, the test file simulates reading data from the sensor.

### 3.4.2 Simulated Data

Using some set values defined at the top of the file including a hardcoded offset and scaling factor, the test file first sets the weight and tares it. Then based on a random number between 0 and 2, it sends either a 0, small weight, or large weight value.

When the test file calls to the read weight functions, some small noise of $\pm 1g$ is used simulate the variability of the sensor reading. The sensor sends data in 24 bit samples, so the test code also includes the 24 bit shifting. Once the simulated data is created, the main function compares it to the expected result. If it's within one gram of the expected low high or zero value, then the value passes. This tolerance is arbitrary for the testing and will be different for the production code.

# A Problem Formulation

## A.1 Conceptualizations for Design Approach

### A.1.1 Contactless Capacitive Water Level Sensing

A contactless sensor is a small puck that attaches to the side of the water filter. It would work by detecting when the level goes below a certain point, and sending that information to the microcontroller. It's a simple and easy way to do one level detection. It produces a binary output of either above or below the detected level.

**Pros**

- Low-cost
- No contact
- Small mechanical approach

**Cons**

- Need a good environment for sensor, not humid so not great inside a fridge
- Users need to install device themselves, making it hard to standardize what's considered "empty"
- Binary output doesn't allow for percentage based information, or any form of consumption metrics.

Because of the noted cons to this design style, we decided to find another solution. The user needing to place the sensor on their own device could throw off any configuration we could complete on our side. It puts too much onto the user for it to work well.

### A.1.2 Weight based, Load

The design places the pitcher unto a load cell to measure the weight. We will calibrate it to the changes in weight.
**Pros**

- Highest accuracy of all designs
- Fits to many pitcher dimensions

**Cons**

- Need a good environment for sensor, not humid so not great inside a fridge

### A.1.3 Ultrasonic

An ultrasonic sensor estimates the water level by measuring the echo time of sound waves reflected from the water surface. The sensor would be installed attached to the side of the filter as close to the top of the water compartment.
**Pros**

- Inexpensive
- Easy implementation

**Cons**

- Humidity problem
- Mounting hardware and wiring sensor is difficult without modifications to container.

If there are modifications to the filter itself in order for a design to work, then the design could not be possible to produce. If we chose this design and needed to modify the pitcher to mount the hardware or get the wiring right, then we would have to sell pre-modified pitchers as well, otherwise users might not be able to use the device properly. Because of these reasons we decided against using this design style.

### A.1.4 Laser

A laser or time-of-flight sensor measures the distance from the sensor to the water surface. The measured distance is converted into liquid height and volume.
**Pros**

- Continous measurements
- Non contact

**Cons**

- Humidity problem

- Clear line of sight
- Mounting problems

Some of the same reasons we didn't choose the ultrasonic sensor apply to this laser sensor design. On top of that in order for us to use continuous measurements, the battery power required would raise and we'd have to figure out how to power the device for long periods of time.

## A.2 Brainstorming

Items to decide on:

- Microcontroller
- Sensor
  - Ultrasonic: Pointed down from the lid onto a floating object
  - Load Cell: Place the container on the load cell to measure the weight and calculate the water level
  - Laser: Shoots a laser into the water from the lid, and the distance is sent back to the sensor
  - Camera: Setup to watch the water filter and measure the water level based on computer vision
  - Non-Contact Sensor: Patch that sits on the side, when water level goes below sensor it can alert the device
- Server
  - HTTP
  - AWS
  - Vercel
- Challenges
  - Signal in the fridge
  - Size limit
  - Contact Vs No Contact
  - Wifi connectivity
  - Power source (battery, USB, etc.)

## A.3 Decision Table

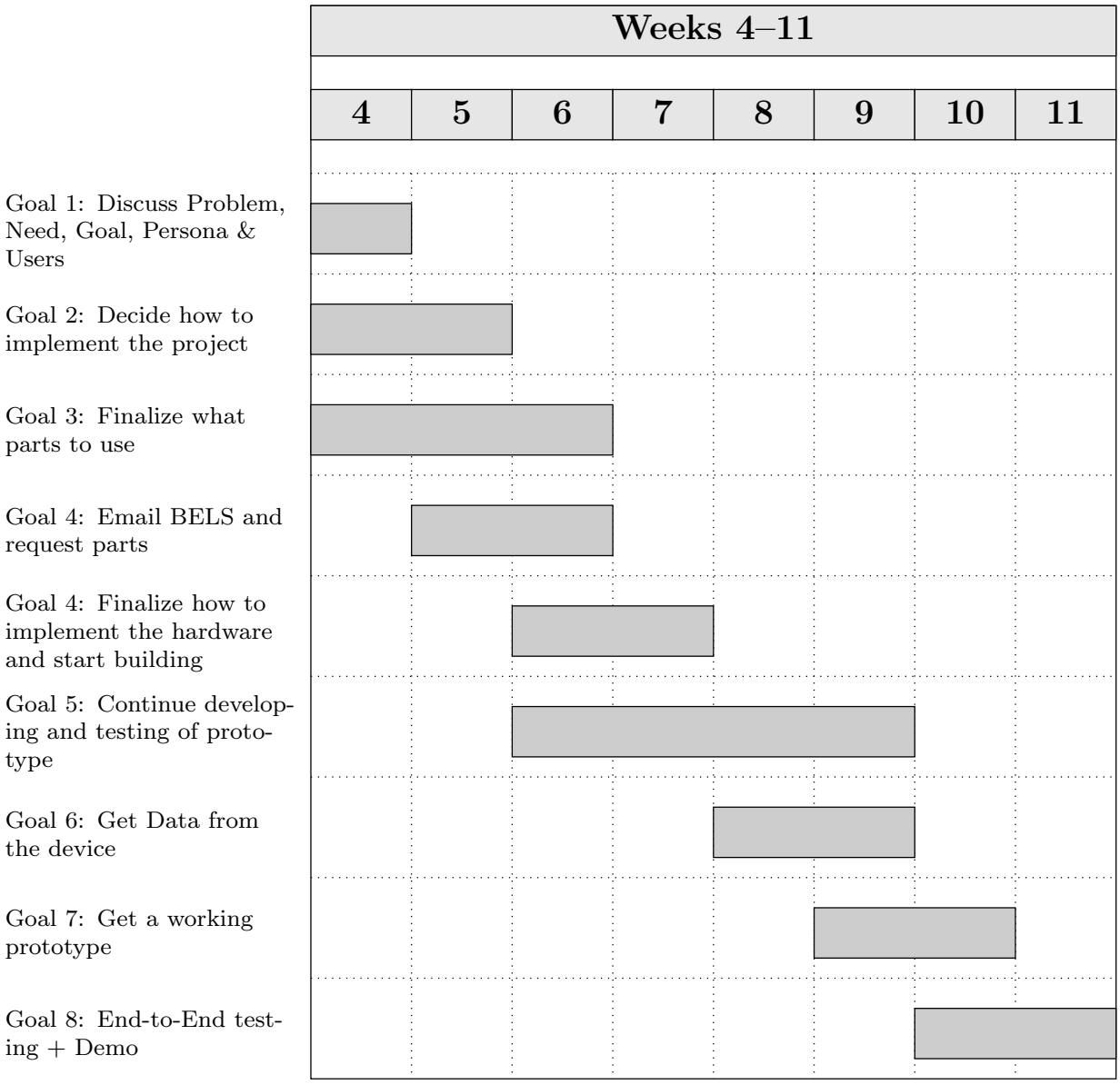
Based on the decision matrix, the weight sensor is the best sensing method because it has the highest weighted total score (4.55). It performs especially well in measurement quality while still keeping cost, power, and overall complexity at a strong level compared to the other options.

| Criteria (Weight) | Capacitive | Weight | Laser | Ultrasonic |
|---|---|---|---|---|
| Cost (0.30) | 5 | 5 | 3 | 5 |
| Power (0.20) | 5 | 4 | 3 | 4 |
| Complexity (0.25) | 3 | 4 | 2 | 3 |
| Measurement Data (0.25) | 3 | 5 | 3 | 3 |
| **Weighted Total** | **4** | **4.55** | **2.70** | **3.85** |

Table 1: Decision matrix comparing feasible sensing approaches (1 = poor, 5 = excellent)

# B Planning

## B.1 Basic Plan / Gantt Chart



## B.2 Division of Labor

**Dev Chodavadiya**: I focused on the physical hardware design and documentation. For hardware, I designed and 3D-printed the base plates for the load cell mount in the BELS lab, iterating on multiple versions to fix flatness and hole alignment issues. I also created the CAD models and engineering drawings for all of the 3D-printed parts. On the documentation side, I contributed to writing and organizing the design document and weekly status reports.

**Logan Bossuwe**: I handled the Load Cell and HX711 sensor hardware to the microcontroller. I soldered and worked on the plate assembly. For the software, I implemented collecting 24bit readings from the HX711 sensor and calibrated it to output grams to be sent via HTTP. I reduced raw reading noise by average values. In addition, I contributed to documentation with conceptualizing design ideas with a decision table and focused on proper documentation of HX711 sensor. I also organized the microconntroller's code to integrating sensor readings with the HTTP POST and wifi provisioning.

**Pieter Nauwelaerts**: My main contributions came in the form of test code for the microcontroller. I created the test system code that would emulate the physical sensor in order to test the controller logic. I also created CAD diagrams to represent the system in its many forms as we designed and tested, figuring out what worked best and what we could expect to present at the end of the term. I contributed to the design document including sections on the microcontroller code design, and the appendix sections about initial concepts. I also prepared the battery system to power our prototype, but ultimately did not implement it as it would conflict with the initial presentation which provides power through the USB connection.

**Reid Graham**: I mainly worked on the microcontroller code and hardware. I helped with physical assembly of the device, including soldering and plate assembly. On the software side, I implemented the WiFi provisioning system, which allows the device to host a Wifi access point and an HTTP server for users to input their WiFi credentials. I also implemented the HTTP functionality for the microcontroller, which allows it to send weight data to the web application.

In the overall project, I helped to organize our planning using the GitLab repository and the general outline of the design document. I also contributed to the planning the higher level organization of the system through the architecture diagram and organizing the work into discrete modules.

**Sam Lai**: I mainly worked on the web application development and testing. Specifically, I implemented the notification system that allows users to receive alerts on their laptops and phones when the water level of the water pitcher is low. I created the UI that allows users to subscribe to notifications on the frontend and also created a table in Supabase to store user subscription data. I implemented two API endpoints to handle subscription management and notification sending. Additionally, I assisted Shriyan in testing the overall web application by creating and conducting unit tests for each functional part on the UI to ensure everything works as intended and is reliable. In terms of the design document,

I contributed to writing and organizing the web application code and test sections. I also proposed the idea of using a weight sensor and wrote down brainstorming ideas for the design.

**Shriyan Gote**: I mainly worked on the web application frontend and deployment. I designed and implemented the Brita water level dashboard UI, including the pitcher graphic, animated water fill, percentage display, last-updated time, and calibration controls (calibrate empty/full) that store values in Supabase. I set up Supabase Realtime so the page updates when new sensor data is posted, and wrote C code to test the server connection using POST HTTP requests from the microcontroller. I also wrote documentation for the web app code and configured deployment (Vercel, environment variables) and kept the repository in sync between the UCSC GitLab and GitHub remotes.

## B.3 Collaboration

Using Gitlab, tasks are self assigned on the issue board that are closely related to our division of labor. From our individual contributions, we have weekly meetings for everyone to catchup on what we did in person and online using discord. Whenever individuals tasks have overlap or dependency need we collaborate on the task. Tests plans/code are made in communication with the original maker of a task.

# C Test Plan & Results

## C.1 Overview

There are multiple sections of our design that require testing individually, then together as a whole. The following list represents the pathway we will take when testing our prototype segments.

- MCU Data Reporting
- MCU Web Posting
- Vercel Receiving and Displaying Data
- Notification System
- Phone App Integration
- Prototype Setup Out-Of-The-Box

### MCU Data Reporting

The pressure sensors we use to build the base will output data to our MCU device. We want to be sure the MCU is capable of receiving this data and displaying it to the debug terminal, which will be IDF MONITOR. Once we can verify posting to the terminal we will move to the next stage of the data reporting testing.

With data being displayed properly, we can now use this to interpret what an empty versus full pitcher could be expected to weigh. The first test will include to test a known weight to make sure we get accuracate weight readings. Next we will ensure the that when there is no wight on the plate, the data will be read as 0 g. Once this correct, we will move on to the more complex calibration with Pitcher + water.

We will use different types of containers given the variety of containers possible to be used by customers. We will test at different set water levels, and verify that the expected weight represents the following formula.

$$W_{total} = W_{container} + V * \frac{g}{mL} \tag{1}$$

Water weighs 1 gram per milliliter so if we keep track of the volume we put into the container we know exactly what to expect for output weight if we know the base weight of the container alone.

### MCU Web Posting

To test this functionality the simplest way is to have the MCU send some sort of HTTP POST pointed at one of our local machines. We can receive and print this data and then move forward to testing sensor data output.

With a working sensor we can trigger a POST request when an event happens. In this test case it could be picking up the container from the base which triggers a POST request with some set information. Upon successful receipt of this we can confirm that the HTTP method is working on a small scale and move to the next segment of testing.

### Vercel Receiving and Displaying Data

Once we have confirmed the MCU can send POST requests to a local machine, the next stage is to implement that using Vercel.

### Notification System

With data being sent out of the MCU, the next stage is to test if we can send a notification on some set event. For a test case this could be a container being placed on the device. The data sent as part of this notification could be a value representing the weight of the container, or an estimate of how much liquid is inside.

If we know how much liquid is inside the container when we place it on the base, it is easy to verify if the value we receive in the notification is the correct volume. Using different volumes we can create a table to show how close the reported volume was, and tweak our math to get it closer to accurate with different containers.

When we pass the fake data tests, we will compare the debug terminal and the notification water levels to ensure that it is correct.

### Phone App Integration

With the notification system working, we can now test if the notifications are being sent to the phone app. We will use the same test case as before, which is placing a container on the

base. We will verify that the notification is received on the phone and that the data in the notification is correct.

We will also verify that the current fill level of the container is being displayed on the phone app. We will test this by placing a container with a known volume of liquid on the base and verifying that the fill level displayed on the phone app is correct.

#### Prototype Setup Out-Of-The-Box

In order to set up the prototype, the web app must connect to the MCU through its hosted soft AP and scan a QR code to enter the WiFi credentials. To test this, we will try to connect to the MCU on boot multiple times using different devices, such as iOS vs Android, and see if it is still able to connect through soft AP. We will also test different types of WiFi networks, such as mobile hotspots, traditional password-protected networks, and networks with outside authorization such as university WiFi.

The user must also calibrate the device by measuring the empty and full weights of the filter on the weight sensor. On the webapp, they will be given instructions step-by-step to do this. In order to test this process, we will give the device to a new user and see if they are able to clearly follow the instructions and calibrate the device correctly. We will also test robustness by intentionally disobeying the instructions, such as by weighing it empty both times, to see our device handle it.

## C.2 System Testing

### WiFi Capability & Provisioning

**Objective:** Verify that the Smart Water Filter Base can be provisioned with WiFi credentials and reliably connect to a wireless network for communication with external devices.

**Test Cases:**

- **Initial Provisioning Test**
  - **Procedure:** Power on the device in provisioning mode and use the setup interface to enter WiFi SSID and password.
  - **Expected Result:** The device stores the credentials and connects to the specified network.
- **Automatic Reconnection Test**
  - **Procedure:** Power cycle the device after provisioning has been completed.
  - **Expected Result:** The device reconnects automatically to the saved WiFi network without requiring reconfiguration.
- **Invalid Credential Handling**
  - **Procedure:** Attempt provisioning using an incorrect WiFi password.
  - **Expected Result:** The device fails to connect and indicates a connection error or returns to provisioning mode.

- **Network Loss Recovery**
  - **Procedure:** Disconnect the router or move the device out of range, then restore the network.
  - **Expected Result:** The device automatically reconnects once the network becomes available.

**Success Criteria:** The device can be provisioned successfully, stores WiFi credentials correctly, and maintains reliable connectivity to the network during normal operation.

## Group Management Test Plan

**Objective:** Verify that users can join and leave a group associated with a specific Smart Water Filter Base and that group membership is updated correctly in the application.

**Test Cases:**

- **Join Group Test**
  - **Procedure:** A user enters the group code or selects the filter in the application to join its group.
  - **Expected Result:** The user is successfully added to the group and gains access to the filter's information.
- **Leave Group Test**
  - **Procedure:** A user selects the option to leave the group in the application.
  - **Expected Result:** The user is removed from the group and no longer sees the filter's data.
- **Multiple User Membership Test**
  - **Procedure:** Multiple users join the same filter group from different devices.
  - **Expected Result:** All users are successfully added and can view the same filter information.
- **Invalid Group Handling**
  - **Procedure:** Attempt to join a group using an invalid or non-existent group code.
  - **Expected Result:** The application rejects the request and displays an error message.

**Success Criteria:** Users can reliably join and leave groups associated with a filter, and group membership is accurately reflected across all devices.

## Water Level Monitoring and Display

**Objective:** Verify that the Smart Water Filter Base accurately measures the water level and correctly displays this information to users through the application.

**Test Cases:**

- **Sensor Calibration Test**
  - **Procedure:** Empty the container and select the "Empty" calibration option in the application. Then fill the container to the maximum level and select the "Full" calibration option.
  - **Expected Result:** The system stores the calibration values and uses them to correctly scale future sensor readings to the displayed water level.
- **Water Level Accuracy Test**
  - **Procedure:** Fill the container to several known levels and record the value reported by the Smart Water Filter Base.
  - **Expected Result:** The reported level closely matches the actual water level within an acceptable error range.
- **Empty and Full Level Test**
  - **Procedure:** Test the system with the container completely empty and completely full.
  - **Expected Result:** The application correctly displays the minimum and maximum water levels.
- **Data Transmission Test**
  - **Procedure:** Verify that sensor readings are transmitted from the Smart Water Filter Base to the application over WiFi.
  - **Expected Result:** The application consistently displays the most recent water level data.

**Success Criteria:** The Smart Water Filter Base accurately measures water level and reliably communicates this information to the user interface.

## Notification System Test Plan

**Objective:** Verify that the system generates and delivers notifications to users when specific conditions occur, such as low water levels or filter maintenance requirements.

**Test Cases:**

- **Low Water Level Notification**
  - **Procedure:** Reduce the water level below the defined threshold.
  - **Expected Result:** A notification is generated and displayed to users indicating that the water level is low.
- **Notification Delivery Test**
  - **Procedure:** Trigger a notification event and verify that it appears in the application interface.
  - **Expected Result:** The notification is delivered and visible to the user.

- **Multiple User Notification Test**
  - **Procedure:** Trigger a notification event while multiple users are members of the same group.
  - **Expected Result:** All users in the group receive the notification.

**Success Criteria:** Notifications are reliably generated and delivered to all relevant users when system conditions trigger an alert.

# D Review

## D.1 Team Reflections

**Dev Chodavadiya**: I think the project went well overall. The team did a good job splitting up the work and communicating when things needed to come together. On my end, the 3D printing process took more iterations than expected to get the plates flat and properly aligned, but working through those issues taught me a lot about designing for manufacturability. If I were to do it again, I would start the CAD and printing process earlier to leave more room for iteration, and I would also try to get more involved with the software side of the project to better understand the full system end to end.

**Logan Bossuwe**: I think the development of the project went well with our organization of tasks and documentation. Individually each person contributed on what was said to work towards while being proactive with documentation. If I were to do it again, I would properly plan out ideas of a design with load cell for conceptualisations. We jumped to the conclusion it was the best choice but did not properly document ideas with plates so it was slow to start. I would have also have contributed more the web development side as I want add features to gain more experience with it.

**Pieter Nauwelaerts**: This project development went very smoothly. The team met and had very productive meetings with lots of discussion about how we wanted to approach problems. We all came together and implemented what we thought was best as a group and I think our product really demonstrates that. I wish I could have had more time to spend working closer with the web app design side because that was the newest concept to me, but I look forward to next quarter where we will hopefully get more experience and time working on that side of the project. I think if we had started developing the board sooner I could have had a working PCB design so we could move away from the current microcontroller setup, but it will be a complete design by the end of the project time.

**Reid Graham**: I think development went well. We were able to divide the work evenly between ourselves, individually work on our submodules, and combine them into the prototype. One thing I would have done differently would have been to plan out our organization of the repository and design document earlier, since we had to move around some scattered

parts to create the finalized versions. I would also try to contribute more to the web development side, since I would like more experience with it and could implement additional features.

**Sam Lai**: I think the execution of our project went pretty well overall. We were able to collaborate effectively to complete tasks on the hardware and software sides to meet our milestones. If I were to do it again, I would try to start development earlier because we had some delays during the planning and brainstorming phase. I would also try to take up some more of the work in other areas of the project because I was mostly focused on the web application, which was not a major focus for the prototype.

**Shriyan Gote**: I think the project went well. I enjoyed building the web dashboard and calibration flow and getting Realtime updates working so the UI refreshes when new data comes in. Working across the stack—frontend, Supabase, and C code to test the server—gave me a clear picture of how the pieces fit together. If I were to do it again, I would set up the two remotes (UCSC and GitHub) and pull/push workflow earlier to avoid merge conflicts, and I would document the deployment steps (Vercel, env vars) as we went so the rest of the team could deploy more easily.